

Chapter 1 Lab – Text as Data: TF-IDF and Document Similarity on SEC Filings

Advanced Machine Learning for Finance

Giovanni Della Lunga

University of Bologna

April–May 2026

Lab Objectives

This notebook applies the concepts from Lesson 1 to **real financial text** drawn from the SEC EDGAR database:

- ➊ **Corpus construction** – fetch 10-K annual filings from EDGAR for companies across different sectors.
- ➋ **Preprocessing** – compare classical tokenization with subword (BPE) tokenization; examine the effect of stop-word removal, stemming, and lemmatization.
- ➌ **TF-IDF representation** – build sparse vectors using scikit-learn and inspect distinctive terms.
- ➍ **Document similarity** – compute and visualize cosine similarity; compare it with Euclidean distance.
- ➎ **Critical analysis** – identify where TF-IDF succeeds and where it fails.

Roadmap

- 1 Building a Corpus from SEC EDGAR
- 2 Preprocessing
- 3 TF-IDF Representation
- 4 Document Similarity
- 5 Critical Analysis
- 6 Summary and Connections

SEC EDGAR: The Data Source

- The U.S. Securities and Exchange Commission maintains **EDGAR** (Electronic Data Gathering, Analysis, and Retrieval).
- EDGAR is the main public repository for corporate regulatory filings in the United States.
- Researchers can access annual reports (Form 10-K), quarterly reports (Form 10-Q), and many other disclosures.
- In this lab we use the Python library `EdgarTools` to interact with EDGAR programmatically.

Focus section

We extract the **Risk Factors** section (Item 1A) from each 10-K filing — a narrative section rich in domain-specific language.

Company Selection

Ticker	Company	Sector
JPM	JPMorgan Chase	Banking
GS	Goldman Sachs	Banking
BAC	Bank of America	Banking
AAPL	Apple	Technology
MSFT	Microsoft	Technology
GOOGL	Alphabet	Technology
XOM	ExxonMobil	Energy
CVX	Chevron	Energy
PFE	Pfizer	Healthcare
JNJ	Johnson & Johnson	Healthcare
KO	Coca-Cola	Consumer Staples
PG	Procter & Gamble	Consumer Staples

Design: 12 companies $\times$ 5 sectors. Same-sector companies should cluster together under TF-IDF similarity; cross-sector pairs should not.

Fetching Filing Metadata from EDGAR

```
def get_latest_10k_url(cik):  
    """Fetch the URL of the most recent 10-K  
       filing for a given company CIK."""  
    url = f"https://data.sec.gov/submissions/CIK{cik:010d}.json"  
    resp = requests.get(url, headers=HEADERS)  
    resp.raise_for_status()  
    data = resp.json()  
    recent = data['filings']['recent']  
    for i, form in enumerate(recent['form']):  
        if form == '10-K':  
            acc = recent['accessionNumber'][i].replace('-', '')  
            doc = recent['primaryDocument'][i]  
            return (f"https://www.sec.gov/Archives/edgar/"  
                    f"data/{cik}/{acc}/{doc}",  
                    recent['accessionNumber'][i])  
    return None, None
```

Key point

The SEC API requires a valid User-Agent header and rate-limiting (max 10 requests/second).

Extracting Risk Factors Text

```
from edgar import Company

def extract_text_from_filing(cik, max_chars=None):
    """Extract Risk Factors (Item 1A) from the latest 10-K."""
    company = Company(str(cik))
    filing = company.get_filings(form="10-K").latest()
    tenk = filing.obj()
    text = getattr(tenk, "risk_factors", None)
    if not text:
        text = tenk["Item 1A"] # fallback
    text = re.sub(r"\s+", " ", str(text)).strip()
    return text[:max_chars] if max_chars else text
```

Corpus statistics

12 documents, ranging from ~34 000 chars (CVX) to ~142 000 chars (GS). Approximate token counts: 5 000–21 000 tokens per document.

Classical vs. BPE Tokenization

Classical tokenization

Split on whitespace and punctuation;
lowercase.

jpmorganchase, must, periodically,
submit, a, detailed, resolution, plan,
to, the, federal, reserve, ...

→ 39 tokens

BPE tokenization (GPT-4)

Subword segmentation using byte-pair
encoding.

J, PM, organ, Ch, ase, must,
periodically, submit, a, detailed,
resolution, plan, ...

→ 44 tokens

Observation

BPE splits financial terms into semantically meaningless fragments: FDIC → FD|IC, EBITDA → EB|IT|DA. This is one reason why LLMs struggle with arithmetic and precise entity matching.

The Preprocessing Pipeline

```
stemmer    = PorterStemmer()
lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))

def preprocess(text, remove_stops=False,
               do_stem=False, do_lemma=False):
    tokens = word_tokenize(text.lower())
    tokens = [t for t in tokens if t.isalpha() and len(t) > 2]
    if remove_stops:
        tokens = [t for t in tokens if t not in stop_words]
    if do_stem:
        tokens = [stemmer.stem(t) for t in tokens]
    elif do_lemma:
        tokens = [lemmatizer.lemmatize(t) for t in tokens]
    return ' '.join(tokens)
```

We build **four** versions of the corpus: raw, no_stopwords, stemmed, lemmatized.

The Effect of Preprocessing Choices

Version	Vocab size	Avg tokens/doc
raw	5,462	8,893
no_stopwords	5,377	6,560
stemmed	3,059	6,560
lemmatized	4,615	6,560

- Each preprocessing step **reduces** the vocabulary size — this is a form of information compression.
- Stemming is the most aggressive single step.
- The question is always: **does the removed information matter for the task?**

Task dependence

For document-level similarity, removing stop words typically helps. For sentiment analysis, removing “not” can invert the meaning of a sentence.

Building the TF-IDF Matrix

We use the **lemmatized, stop-word-free** version of the corpus — the version that best balances vocabulary reduction with semantic preservation.

```
tfidf = TfidfVectorizer(  
    max_features = 5000, # keep top 5000 terms  
    min_df       = 2,    # term must appear in >= 2 docs  
    max_df       = 0.95, # ignore terms in > 95% of docs  
    sublinear_tf = False, # raw TF, to match lecture  
    norm         = 'l2',  # L2 normalization (standard)  
)  
tfidf_matrix = tfidf.fit_transform(docs_for_tfidf)
```

Result

TF-IDF matrix shape: $12 \times 2,584$. Density: $\sim 39.5\%$ — approximately 60 % of entries are zero. This is a **sparse representation**.

Most Distinctive Terms per Company

```
def top_tfidf_terms(matrix, feature_names, doc_index, n=10):  
    row = matrix[doc_index].toarray().flatten()  
    top_idx = row.argsort()[::-1][:n]  
    return [(feature_names[i], round(row[i], 3))  
            for i in top_idx]
```

For each document we extract the terms with the highest TF-IDF weight — terms that are **frequent in that document** but **rare across the corpus**.

The IDF mechanism at work

Terms like *revenue* or *operating* are common across all 10-K filings $\Rightarrow$ low IDF weight. Terms like *drilling* or *pharmaceutical* are sector-specific $\Rightarrow$ high IDF weight $\Rightarrow$ high TF-IDF score.

Interpreting the Top Terms

Expected patterns:

- **Banking** companies share terms such as *loan, deposit, credit, interest*.
- **Technology** companies share terms such as *cloud, user, advertising*.
- **Energy** companies share terms such as *oil, drilling, exploration*.

What to watch for:

- Cross-sector overlap: terms like *regulatory* or *compliance* may appear across sectors.
- Company-specific terms: *iPhone* for AAPL, *Azure* for MSFT.
- The IDF component suppresses terms that are too ubiquitous.

Key idea

TF-IDF automatically surfaces **discriminative** vocabulary — precisely the terms that distinguish one document from another.

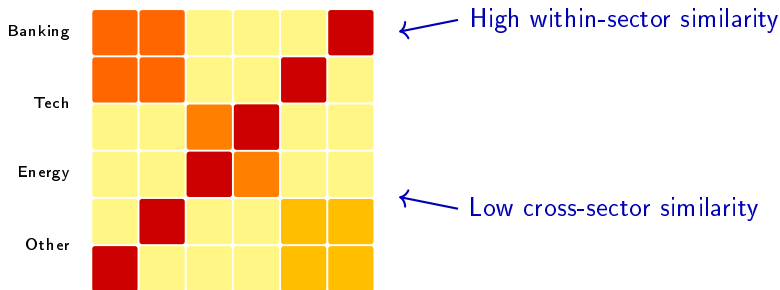
Cosine Similarity Matrix

```
cos_sim = cosine_similarity(tfidf_matrix)
cos_df = pd.DataFrame(cos_sim, index=tickers, columns=tickers)
```

The result is a 12×12 matrix where entry (i, j) records the cosine similarity between the TF-IDF vectors of companies i and j .

- Diagonal entries = 1 (each document is identical to itself).
- Off-diagonal entries $\in [0, 1]$ (all TF-IDF weights are non-negative).
- We visualize this as a **heatmap** with color-coded sector labels.

Heatmap Visualization



Expected pattern

Within-sector blocks show **higher** cosine similarity than cross-sector blocks, confirming that companies in the same industry share domain-specific vocabulary in their Risk Factors.

Do Companies Cluster by Sector?

```
within_sector = []  
across_sector = []  
for i in range(len(tickers)):  
    for j in range(i+1, len(tickers)):  
        sim = cos_sim[i, j]  
        if COMPANIES[tickers[i]]['sector'] == \  
            COMPANIES[tickers[j]]['sector']:  
            within_sector.append(sim)  
        else:  
            across_sector.append(sim)
```

Within sector

Higher average cosine
($n = 11$ pairs)



Across sectors

Lower average cosine
($n = 55$ pairs)

Conclusion

TF-IDF similarity correctly separates same-sector from cross-sector document pairs in this corpus.

Euclidean Distance vs. Cosine Similarity

Recall from the lecture that Euclidean distance conflates **what** a document is about (direction) with **how long** it is (magnitude).

Cosine similarity

- Compares directions only.
- Within-sector pairs score high.
- Length-invariant.

Euclidean distance

- Mixes direction and magnitude.
- A short and a long document on the same topic may appear distant.
- Sector structure is blurred.

Empirical verification

When we compute both metrics on the same TF-IDF matrix, the sector clustering visible under cosine similarity becomes less clear under Euclidean distance — confirming the theoretical argument from the lecture.

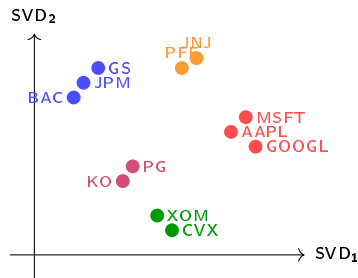
Visualizing the Document Space

We reduce the TF-IDF vectors to 2 dimensions using **Truncated SVD** (related to PCA for sparse matrices).

- Each dot is a company.
- Colors indicate sectors.
- Companies in the same sector cluster together.

What SVD captures

The first two components span the directions of maximum variance in the TF-IDF space. Sector separation along these axes confirms that the vocabulary structure encodes industry identity.



Schematic illustration of the SVD projection

Where TF-IDF Succeeds

- **Same-sector clustering:** companies in the same industry share domain-specific vocabulary (*loan*, *deposit* for banks; *cloud*, *user* for tech firms).
- **Discriminative terms:** TF-IDF automatically surfaces the vocabulary that distinguishes each company or sector from the rest of the corpus.
- **Interpretability:** every dimension of the TF-IDF vector corresponds to a known word, making it easy to explain why two documents are similar.

Practical relevance

TF-IDF remains a strong baseline for document retrieval, topic analysis, and exploratory text mining on financial corpora.

Where TF-IDF Fails

TF-IDF similarity requires **lexical overlap**: two documents can only be similar if they share the same words.

The semantic gap

Sentence A: *The company implemented a cost reduction program.*

Sentence B: *The firm executed an expense cutting initiative.*

A human reader sees these as nearly identical in meaning. After stop-word removal, their TF-IDF vectors share **zero terms** $\Rightarrow$ cosine similarity = 0.

Systematic blind spots include:

- **Synonymy**: “cost reduction” vs. “expense cutting.”
- **Paraphrase**: same idea, entirely different wording.
- **Cross-domain similarity**: two sectors facing structurally similar risks described with different vocabulary.

Demonstrating the Semantic Gap

```
pair_a = "The company implemented a cost reduction program"
pair_b = "The firm executed an expense cutting initiative"

v_a = tfidf.transform([preprocess(pair_a,
                                remove_stops=True, do_lemma=True)])
v_b = tfidf.transform([preprocess(pair_b,
                                remove_stops=True, do_lemma=True)])

sim = cosine_similarity(v_a, v_b)[0, 0]

print(f"Cosine similarity (TF-IDF): {sim:.3f}")
# Output: 0.000
```

The fundamental limitation

TF-IDF captures **lexical co-occurrence**, not **semantic meaning**. This is the core limitation that motivates the transition to **dense representations** (embeddings).

Explaining Similarity: Shared Terms

```
def explain_similarity(t1, t2, matrix, feature_names,
                      tickers, top_n=10):
    v1 = matrix[tickers.index(t1)].toarray().flatten()
    v2 = matrix[tickers.index(t2)].toarray().flatten()
    contrib = v1 * v2 # element-wise (dot-product terms)
    top_idx = contrib.argsort()[::-1][:top_n]
    for idx in top_idx:
        if contrib[idx] > 0:
            print(f" {feature_names[idx]:<20s} "
                  f"product = {contrib[idx]:.4f}")
```

- High-similarity pairs share **content terms**: banking pairs share *loan, credit, deposit*.
- Low-similarity pairs share only **generic terms**: *risk, business, result*.

False Positives and False Negatives

Highest cross-sector similarities

(potential false positives)

- Some cross-sector pairs share regulatory or compliance vocabulary.
- Shared legal boilerplate inflates similarity.
- These are *lexical* connections, not necessarily *semantic* ones.

Lowest within-sector similarities

(potential false negatives)

- Companies in the same sector may use different terminology for the same risks.
- Business model differences within a sector reduce lexical overlap.
- TF-IDF misses the underlying thematic similarity.

Implication

A robust retrieval system must combine lexical matching (TF-IDF / BM25) with semantic matching (dense embeddings) — the **hybrid retrieval** architecture.

What We Have Seen

- ➊ **Real financial text** can be retrieved from SEC EDGAR and transformed into a structured corpus with standard Python tools.
- ➋ **Preprocessing choices matter**: stop-word removal, stemming, and lemmatization each change the vocabulary and the resulting similarity structure. There is no single “correct” pipeline.
- ➌ **TF-IDF + cosine similarity** produces a meaningful document space: same-sector companies cluster together.
- ➍ **Euclidean distance** is less appropriate for text vectors because it conflates direction with magnitude.
- ➎ **TF-IDF has a fundamental limitation**: it captures lexical overlap, not semantic meaning.

Lesson 2 – Dense Representations

We will see how **word embeddings** and **document embeddings** address the semantic gap. The core idea: learn a low-dimensional space where words with similar *contexts* — not just identical *forms* — receive similar vectors.

Notebook 2 – BM25 Retrieval

We return to the retrieval problem and compare TF-IDF retrieval with **BM25** on this same corpus, verifying the saturation and length-normalization improvements discussed in the lecture.

From lexical overlap to semantic proximity

`notebook_01.ipynb` → open in JupyterLab